

Lecture-9. Pygame I

1. Getting Started with Pygame

Pygame is a set of Python modules designed for writing multimedia applications like video games. It provides functionalities like handling images, sounds, events, and user input.

Pygame is a powerful Python library designed for game development. In this lecture, we will cover essential aspects of Pygame that will help you create interactive and visually appealing games.

The topics include working with fonts and text, handling advanced input, implementing centralized scene logic, and creating a simple game.

Installation:

To install Pygame, you can use pip:

```
pip install pygame
```

Basic Setup:

First, initialize Pygame, create a screen, and set a game loop.

```
import pygame # this is needed to access the PyGame framework
import sys # provides system-specific parameters and functions

# Initialize Pygame/It initializes all the modules required for PyGame
pygame.init()

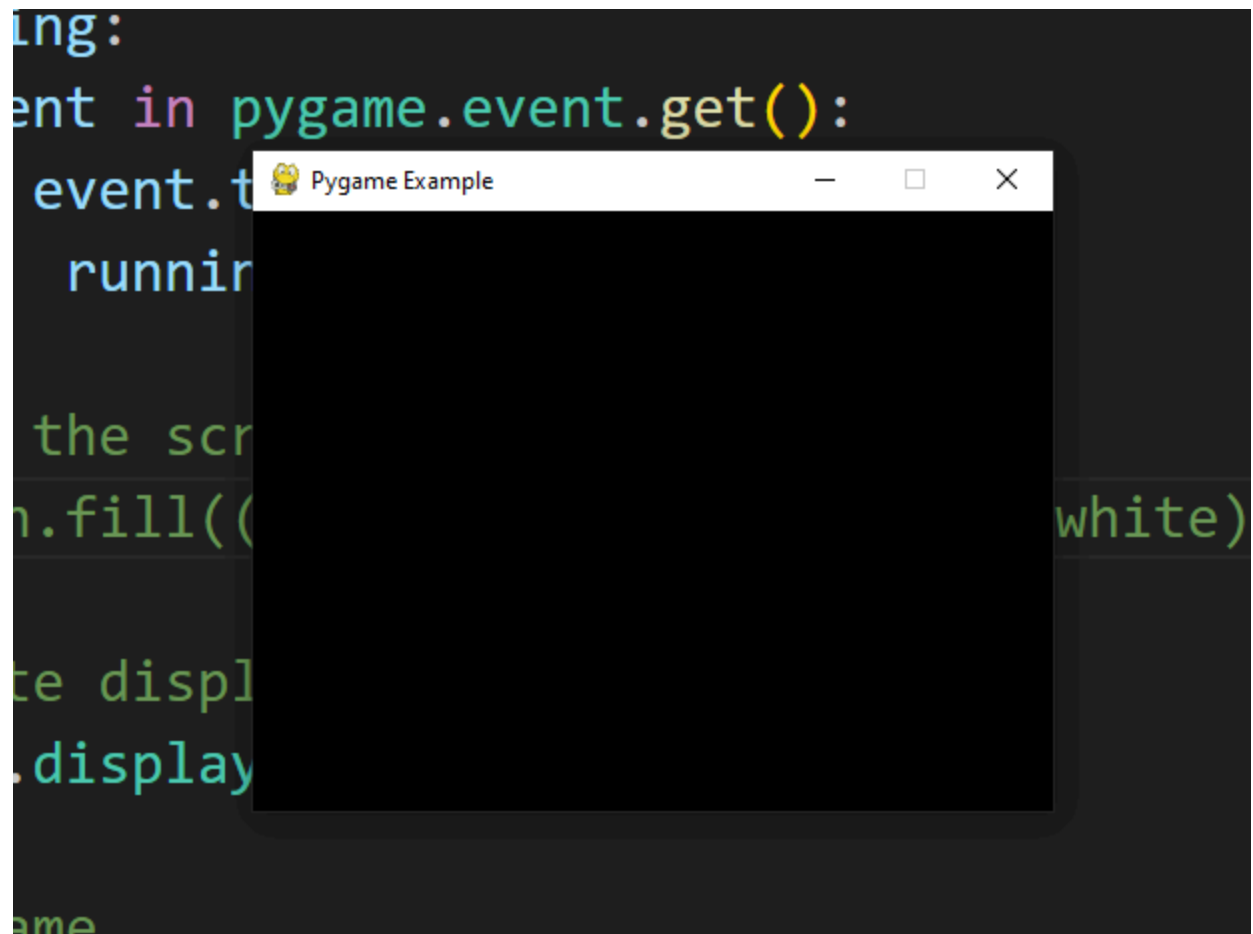
# Set up display
screen = pygame.display.set_mode((400, 300))
pygame.display.set_caption("Pygame Example")
```

```
# Game loop
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    # Fill the screen with color
    screen.fill((255, 255, 255)) # RGB (white)

    # Update display
    pygame.display.flip()

# Quit Pygame
pygame.quit()
sys.exit()
```



2. Working with Images

Pygame allows you to load and manipulate images. Images can be drawn onto the screen or used as sprites for characters.

Loading and Displaying an Image:

```
import pygame
import sys

# Initialize Pygame
pygame.init()

# Set up display
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Image Example")
```

```

# Load an image
image = pygame.image.load('ball.png') # Make sure the image
is in the same directory

# Game loop
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    # Fill the screen with color
    screen.fill((255, 255, 255))

    # Scale the image to fit the screen size (adjust size if
    needed)
    image = pygame.transform.scale(image, (100, 100)) # Resi
    zing it to the full screen size

    # Draw the image on the screen
    screen.blit(image, (10, 10))

    # Update display
    pygame.display.flip()

# Quit Pygame
pygame.quit()
sys.exit()

```

3. Music and Sound Effects

Pygame provides easy integration of sound effects and background music to enhance your game.

Playing Music:

To play music, you need to load it using `pygame.mixer.music` and call `pygame.mixer.music.play()`.

```
import pygame
import sys

# Initialize Pygame and the mixer
pygame.init()
pygame.mixer.init()

# Set up display
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Music and Sound Example")

# Load background music
pygame.mixer.music.load('background_music.mp3')

# Play music (looping)
pygame.mixer.music.play(-1) # -1 for infinite loop

# Game loop
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    # Fill the screen with color
    screen.fill((255, 255, 255))

    # Update display
    pygame.display.flip()

# Quit Pygame
```

```
pygame.quit()
sys.exit()
```

Playing Sound Effects:

You can load and play short sound effects as well.

```
# Load a sound effect
sound = pygame.mixer.Sound('effect_sound.wav')

# Play the sound effect
sound.play()
```

4. Geometric Drawing

Pygame allows you to draw basic geometric shapes like rectangles, circles, and lines directly on the screen.

Example Code: Drawing Shapes

```
import pygame
import sys

# Initialize Pygame
pygame.init()

# Set up display
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Geometric Shapes Example")

# Game loop
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
```

```

# Fill the screen with color
screen.fill((255, 255, 255))

# Draw a rectangle (x, y, width, height)
pygame.draw.rect(screen, (255, 0, 0), (100, 100, 200, 150))

# Draw a circle (x, y, radius)
pygame.draw.circle(screen, (0, 0, 255), (400, 300), 50)

# Draw a line (start_pos, end_pos, color, width)
pygame.draw.line(screen, (0, 255, 0), (100, 100), (400, 400), 5)

# Update display
pygame.display.flip()

# Quit Pygame
pygame.quit()
sys.exit()

```

5. Timer

Pygame provides functionality to track time and manage events based on time intervals using `pygame.time.Clock()` and `pygame.time.get_ticks()`.

Setting a Timer with Pygame:

```

import pygame
import sys

# Initialize Pygame
pygame.init()

```

```

# Set up display
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Timer Example")

# Set up clock
clock = pygame.time.Clock()

# Start timer
start_time = pygame.time.get_ticks()

# Game loop
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    # Get elapsed time
    elapsed_time = pygame.time.get_ticks() - start_time
    print(f"Elapsed time: {elapsed_time / 1000:.2f} seconds")

    # Fill the screen with color
    screen.fill((255, 255, 255))

    # Update display
    pygame.display.flip()

    # Cap the frame rate
    clock.tick(60) # 60 frames per second

# Quit Pygame
pygame.quit()
sys.exit()

```


List of common `event.key` values used in Pygame to detect keyboard input. Here are some of the notable key constants:

Arrow Keys

- `pygame.K_UP` → Up arrow key
- `pygame.K_DOWN` → Down arrow key
- `pygame.K_LEFT` → Left arrow key
- `pygame.K_RIGHT` → Right arrow key

Alphabet Keys

- `pygame.K_a` → 'A' key
- `pygame.K_b` → 'B' key
- `pygame.K_c` → 'C' key

Number Keys

- `pygame.K_0` → '0' key
- `pygame.K_1` → '1' key
- `pygame.K_2` → '2' key

Special Keys

- `pygame.K_RETURN` → Enter key
- `pygame.K_ESCAPE` → Escape key
- `pygame.K_BACKSPACE` → Backspace key
- `pygame.K_TAB` → Tab key
- `pygame.K_SPACE` → Spacebar
- `pygame.K_DELETE` → Delete key
- `pygame.K_HOME` → Home key
- `pygame.K_END` → End key
- `pygame.K_PAGEUP` → Page Up key

- `pygame.K_PAGEDOWN` → Page Down key
- `pygame.K_INSERT` → Insert key

6. Fonts and Text

Text rendering is an essential part of any game, whether for displaying scores, messages, or user instructions.

Loading and Rendering Fonts

- **Initializing Fonts:** Pygame provides a `pygame.font.Font` class to load and display text.
- **Rendering Text:** `render()` method is used to create a surface containing the text.

Example:

```
import pygame
pygame.init()

# Create screen
screen = pygame.display.set_mode((500, 300))

# Load font (None loads default system font)
font = pygame.font.Font(None, 36)

# Render text
text_surface = font.render("Hello, Pygame!", True, (255, 255, 255))

# Game loop
running = True
while running:
    screen.fill((0, 0, 0))
    screen.blit(text_surface, (100, 100))
    pygame.display.flip()
```

```
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        running = False

pygame.quit()
```

Key Points:

- The second argument in `render()` enables anti-aliasing.
 - Text can be displayed using `blit()`.
-

7. More on Input

Handling user input effectively enhances interactivity in games.

Handling Keyboard Input

```
for event in pygame.event.get():
    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_SPACE:
            print("Space key pressed")
```

Handling Mouse Input

```
for event in pygame.event.get():
    if event.type == pygame.MOUSEBUTTONDOWN:
        print("Mouse clicked at", event.pos)
```

Detecting Held Keys

```
keys = pygame.key.get_pressed()
if keys[pygame.K_LEFT]:
    print("Left arrow key is held down")
```

8. Centralized Scene Logic

Managing different scenes in a game (menu, gameplay, game over) efficiently is crucial.

Using a Scene Manager

- Create a base `Scene` class.
- Use different classes for different game states.

```
class Scene:
    def __init__(self):
        pass

    def handle_events(self, events):
        pass

    def update(self):
        pass

    def draw(self, screen):
        pass
```

Switching between scenes:

```
current_scene = MainMenu()
while running:
    current_scene.handle_events(pygame.event.get())
    current_scene.update()
    current_scene.draw(screen)
    pygame.display.flip()
```

Why Centralized Scene Logic in Pygame?

When developing games in Pygame, structuring your code effectively is essential for maintainability, scalability, and readability. **A centralized scene logic** approach

helps manage different parts of the game efficiently.

Centralized Scene Logic is a design pattern where the game flow is managed by a **single main controller (scene manager)** that handles transitions between different game states (menus, gameplay, game over, etc.).

Instead of writing separate loops for each game state, we encapsulate logic into different **scene classes** and control them from a single **scene manager**.

Why Use Centralized Scene Logic?

1. Better Organization & Scalability

- Keeps game logic **modular** and **separated** (menu, gameplay, pause, etc.).
- Makes it easier to add new features without modifying the entire game loop.

2. Easier Debugging & Maintenance

- Instead of searching through multiple files for logic, everything is structured in distinct classes.

3. Smooth Transitions Between Scenes

- Switching between game states (e.g., from **Menu** → **Game** → **Game Over**) is managed efficiently.
- Avoids spaghetti code where scene changes are scattered throughout the codebase.

4. Encapsulation & Reusability

- Each scene class handles its own rendering and events.
 - Common functionalities (e.g., background music, UI updates) can be shared.
-

Game Creation

Finally, let's integrate everything into a simple game where a player moves a rectangle using arrow keys.

```

import pygame
pygame.init()

# Constants
WIDTH, HEIGHT = 500, 500
WHITE = (255, 255, 255)
BLUE = (0, 0, 255)

# Create screen
screen = pygame.display.set_mode((WIDTH, HEIGHT))
clock = pygame.time.Clock()

# Player settings
player = pygame.Rect(200, 200, 50, 50)
speed = 20 # Move 20 pixels per press
running = True
while running:
    screen.fill(WHITE)

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    # Get pressed keys
    keys = pygame.key.get_pressed()

    # Corrected movement logic with exact boundary checks
    if keys[pygame.K_LEFT] and player.x - speed >= 0:
        player.x -= speed
    if keys[pygame.K_LEFT] and player.x - speed < 0:
        player.x = 0
    if keys[pygame.K_RIGHT] and player.x + speed <= WIDTH - player.width:
        player.x += speed
    if keys[pygame.K_RIGHT] and player.x + speed > WIDTH - player.width:
        player.x = WIDTH - player.width

```

```

er.width:
    player.x = WIDTH - player.width
    if keys[pygame.K_UP] and player.y - speed >= 0:
        player.y -= speed
    if keys[pygame.K_UP] and player.y - speed < 0:
        player.y = 0
    if keys[pygame.K_DOWN] and player.y + speed <= HEIGHT - player.height:
        player.y += speed
    if keys[pygame.K_DOWN] and player.y + speed > HEIGHT - player.height:
        player.y = HEIGHT - player.height

# Draw player
pygame.draw.rect(screen, BLUE, player)

pygame.display.flip()
clock.tick(30) # Limit to 30 FPS
pygame.quit()

```